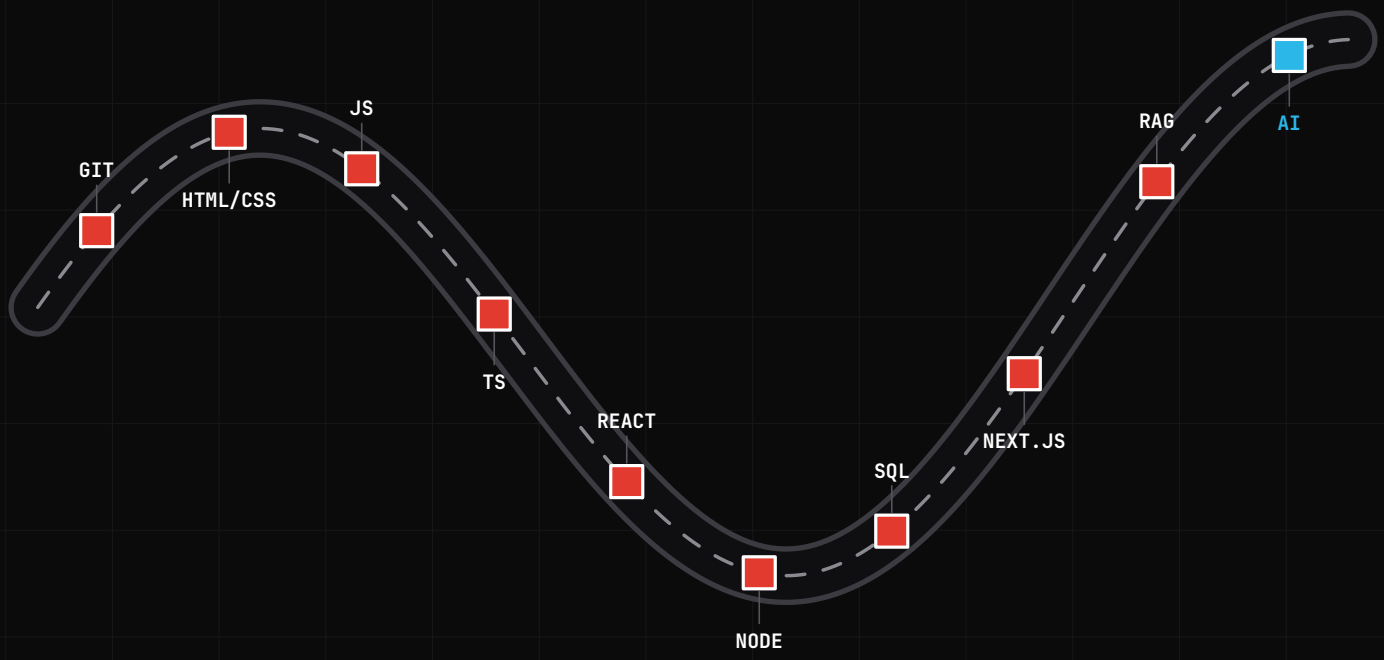
MINDFLARE / IT DEPARTMENT SOCIETY

THE COMPLETE WEB DEVELOPER ROADMAP



From zero to an AI-ready web developer in 12-18 months.

Front-end / Back-end / Full-stack / AI: RAG, chatbots, agents, ML models

MINDFLARE

where algorithm sparks intuition

2026 EDITION

About Mindflare

MINDFLARE

where algorithm sparks intuition

Mindflare is the official society of the IT department. We exist to promote algorithms and competitive programming: we host weekly contests, keep leaderboards, and help students turn problem-solving practice into real skill.

Strong programmers do not stop at puzzles. They build things people use. This roadmap extends the Mindflare philosophy from the contest arena to the web: learn the fundamentals deeply, build real projects, and use modern tools, including AI, with understanding instead of blind copy-paste.

Use it as a map, not a rulebook. Pick your pace, share your progress with the society, and help the people around you move forward. The best way to learn is to build, and the best way to build is together.

> **HOW TO PARTICIPATE**
> Join our weekly contests, post your project demos with the community, and compare your growth on the leaderboard. Consistency compounds.

Table of Contents

About Mindflare 2

Introduction 5

Web Development 6

Roadmap at a Glance 7

Recommended Stack (2026) 8

Part 01 Foundations 10

- How the Web Works & Dev Tooling 11
- Git & GitHub 11
- HTML 12
- CSS 12
- JavaScript 13
- Data Structures, Algorithms & Competitive Programming 15

Part 02 Modern Front-end 16

- TypeScript 17
- React 18
- Tailwind CSS 19
- Web Performance & Accessibility 20

Part 03 Back-end Development 21

- Node.js 22
- Express.js & REST API Design 22
- PostgreSQL & SQL 24
- MongoDB & Redis 26
- Authentication & Security 26

Part 04 Full-Stack Engineering 28

- Automated Testing 29
- Next.js 29
- Docker, CI/CD & Deployment 31

Part 05 AI for Web Developers 32

- AI-Assisted Development 33

© 2026 Mindflare · IT Department Society

LLM Fundamentals & APIs 33
Building AI Chatbots 34
Embeddings, Vector Search & RAG 35
Agents, Tool Use & MCP 37
ML Models in Websites 38
Production AI: Evals, Safety & Cost 39

Part 06 Beyond the Roadmap 41
Design Patterns & Clean Code 42
Additional Skills for Senior Levels 42
Capstone Projects 43
Portfolio, Resume & Placement Prep 44

Suggested Timeline 45

Introduction

Web development changed a lot in the last two years. The classic skills (HTML, CSS, JavaScript, a framework and a database) are still the foundation, but employers now also expect you to ship AI features: chatbots that answer from your own documents, smart search, and models that run inside the product. This guide gives you one structured path through both the foundations and the new layer on top.

It is designed so you always know what to learn next, how long it should take, which concepts matter, which free resources to use, and what to build to prove you understood it.

Target Audience

- Beginners who want to know exactly what to learn to land a web developer role or internship.
- Students who already solve DSA problems and want to turn that strength into real products.
- Developers who want to level up, fill gaps, or add AI features such as RAG and chatbots to their toolkit.

How to Use This Roadmap

- Follow the parts in order. Each part builds on the one before it.
- Spend roughly 30% of your time learning and 70% building. Reading alone does not stick.
- Finish at least one project before moving on from each topic. Project ideas are ordered from simple to complex.
- Time estimates assume steady part-time study alongside college (roughly 15-20 hours a week). Adjust to your schedule.
- Run DSA practice in parallel from the first week. It is not a separate phase.
- Push everything to GitHub. Your commit history is your proof of progress.

Resources

Every topic lists free, official or widely trusted learning resources. Prefer official documentation whenever you can; it is the most accurate and the most up to date. Courses and videos are great for orientation, but the docs are where professionals live.

> **KEEP IT FRESH**
> Tools in this field change fast. Version-specific details here reflect the landscape at the time of writing (late 2026). Always confirm current features, pricing and best practices in the official docs before you commit to a tool.

© 2026 Mindflare · IT Department Society

Web Development

Web development is traditionally split into two areas. Full-stack development combines both. In this roadmap we add a third layer that is now part of the job: AI engineering for the web.

Area	What it covers	Examples
Front-end	Everything users see and touch in a browser: layout, interaction, accessibility, speed.	HTML, CSS, JavaScript, React, Tailwind
Back-end	Everything behind the scenes: servers, APIs, data, authentication, security.	Node.js, Express, PostgreSQL, Redis
Full-stack	Both sides together, plus testing, deployment and automation.	Next.js, Docker, CI/CD
AI engineering	Adding language models, retrieval, agents and ML models to products safely.	LLM APIs, RAG, chatbots, Transformers.js

Where to Start

Start with the foundations (the web itself, Git, HTML, CSS and JavaScript). You get visible results in the browser quickly, which keeps motivation high, and JavaScript is the single language you will carry across the browser, the server and most AI SDKs. After that, move into modern front-end, then back-end, then combine them into full-stack apps, and finish with the AI layer. By the time you add AI features, you will understand every part of the system they plug into.

Why JavaScript and TypeScript?

- One language across browser, server and tooling means you learn concepts once instead of learning syntax twice.
- The job market for JavaScript and TypeScript developers is huge, from startups to large product companies.
- Most modern AI web tooling is TypeScript-first: Vercel AI SDK, LangChain.js, Transformers.js, MCP SDKs and Next.js.
- TypeScript's type system catches whole classes of bugs early, and it makes AI-generated code safer to review.

> **TWO LANGUAGES, ONE PLAN**
> Python is still the home of machine learning. In Part 05 you will learn just enough Python to build and serve ML models. And keep a second language (C++, Java or Python) for competitive programming contests.

© 2026 Mindflare · IT Department Society

Roadmap at a Glance

Five phases, one path. Weeks are the focused learning time for each skill; DSA practice runs alongside everything else.

Skill	Time required	Level
PHASE 1 Foundations (Part 01)	**16 weeks**	
Web basics & dev tooling	1 week	Beginner
Git & GitHub	1 week	Beginner
HTML	2 weeks	Beginner
CSS	4 weeks	Beginner
JavaScript	8 weeks	Beginner
PHASE 2 Modern Front-end (Part 02)	**14 weeks**	
TypeScript	3 weeks	Intermediate
React	8 weeks	Intermediate
Tailwind CSS	3 weeks	Intermediate
PHASE 3 Back-end (Part 03)	**18 weeks**	
Node.js	2 weeks	Intermediate
Express.js & REST APIs	4 weeks	Intermediate
PostgreSQL & SQL	5 weeks	Intermediate
MongoDB & Redis	3 weeks	Intermediate
Authentication & Security	4 weeks	Intermediate
PHASE 4 Full-Stack Engineering (Part 04)	**13 weeks**	
Automated Testing	4 weeks	Advanced
Next.js	6 weeks	Advanced
Docker, CI/CD & Deployment	3 weeks	Advanced
PHASE 5 AI for Web Developers (Part 05)	**16 weeks**	
LLM fundamentals & APIs	3 weeks	Advanced
Building AI chatbots	2 weeks	Advanced
Embeddings, vector search & RAG	4 weeks	Advanced
Agents, tool use & MCP	2 weeks	Advanced
ML models in websites	3 weeks	Advanced
Production AI: evals, safety & cost	2 weeks	Advanced
PARALLEL DSA & competitive programming	**8 weeks core + ongoing**	**Throughout**
Total (excluding parallel DSA)	**77 weeks (about 18 months)**	

Faster route (about 12 months): if you already know the basics of programming, can study 25+ hours a week, or skip optional topics (MongoDB, Docker details), you can compress the plan. Do not skip Git, JavaScript, TypeScript, React, SQL or the AI fundamentals.

© 2026 Mindflare · IT Department Society

Recommended Stack (2026)

You do not need to learn every alternative. Master this core stack first; the alternatives are listed so you recognise them when you meet them.

Layer	Learn this	Good to know
Language	TypeScript (on JavaScript)	Python for ML work; C++ or Java for contests
Front-end	React + Vite or Next.js	Vue, Svelte, Astro, Remix / React Router
Styling & UI	Tailwind CSS + shadcn/ui	CSS Modules, Sass, Radix UI
State & data	TanStack Query, Zustand, React Hook Form + Zod	Redux Toolkit, SWR, tRPC
Back-end	Node.js + Express	Fastify, Hono, NestJS, Bun, Deno
Database	PostgreSQL with Prisma or Drizzle	MySQL, SQLite, MongoDB
Cache & queues	Redis, BullMQ	RabbitMQ, Kafka
Auth	Auth.js or Better Auth, OAuth, JWT/sessions	Passport, Clerk, Keycloak
Testing	Vitest, Testing Library, Playwright, Supertest	Jest, Cypress
DevOps	Git, Docker, GitHub Actions	Kubernetes, Terraform
Hosting	Vercel, Render, Railway or Fly.io	AWS, Google Cloud, Azure, Cloudflare
LLM access	Vercel AI SDK + a hosted model API	Ollama for local models; LangChain.js
Vector search	PostgreSQL + pgvector	Qdrant, Chroma, Pinecone, Weaviate
ML in the browser	Transformers.js, ONNX Runtime Web	TensorFlow.js, MediaPipe
ML on the server	FastAPI + scikit-learn / PyTorch	Hugging Face Inference, Replicate, Modal
Agents & tools	Tool calling, MCP servers in TypeScript	LangGraph.js, Mastra

© 2026 Mindflare · IT Department Society

© 2026 Mindflare · IT Department Society

PART

01 Foundations

Everything else stands on this. Learn how the web works, get comfortable with the terminal and Git, then build real pages with HTML, CSS and JavaScript. Start DSA practice in parallel from week one.

IN THIS PART

How the Web Works & Dev Tooling	1 week
Git & GitHub	1 week
HTML	2 weeks
CSS	4 weeks
JavaScript	8 weeks
Data Structures, Algorithms & Competitive Programming	8 weeks core, then ongoing

© 2026 Mindflare · IT Department Society

How the Web Works & Dev Tooling

Before writing code, understand what happens when you type a URL and press Enter. This mental model makes every later topic (APIs, caching, security, performance) easier to grasp.

TIME REQUIRED	PHASE / LEVEL
1 week	**Beginner**

LEARN WITH MDN: How the web works · The Odin Project

ESSENTIAL CONCEPTS

- **Client-server model:** browsers, servers, requests, responses
- **HTTP:** methods (GET, POST, PUT, PATCH, DELETE), status codes, headers, cookies, caching headers
- **Networking basics:** URLs, DNS, IP, ports, TLS/HTTPS, CDNs
- **The browser:** parsing HTML/CSS, the render pipeline, the JavaScript engine
- **DevTools:** Elements, Console, Network, Application, Performance and Lighthouse panels
- **Terminal:** cd, ls, mkdir, cp, mv, rm, cat, grep, pipes, environment variables
- **Editor setup:** VS Code, extensions (ESLint, Prettier), debugger, snippets
- **Node.js tooling:** install Node through a version manager (nvm or fnm), npm vs pnpm, package.json

MINDFLARE TIP

Open the Network tab on any site you use daily and read the requests. Ask: who sent this, what did it carry, why this status code?

Git & GitHub

Git tracks every change to your code and lets teams work in parallel without overwriting each other. GitHub hosts your repositories and becomes your public portfolio. Use Git from your very first project.

TIME REQUIRED	PHASE / LEVEL
1 week	**Beginner**

LEARN WITH Pro Git (free book) · GitHub Docs

ESSENTIAL CONCEPTS

- **Setup:** init, clone, config, SSH keys
- **Staging & commits:** status, add, restore, commit, amend, .gitignore
- **Inspect & compare:** log, diff, show, blame
- **Branching:** branch, switch, merge, rebase, resolving conflicts
- **Remotes:** remote, fetch, pull, push, upstream tracking
- **Temporary work:** stash, cherry-pick, reset vs revert
- **GitHub workflow:** fork, pull requests, code review, issues, projects
- **Good habits:** small commits, meaningful messages (Conventional Commits), README files, branch protection

MINDFLARE TIP
Commit daily. A green contribution graph built on real projects is the cheapest credibility you can earn.

HTML

HTML gives web pages their structure and meaning. Good HTML is semantic: it tells browsers, search engines and screen readers what each part of the page actually is.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Beginner**

LEARN WITH MDN HTML guide · web.dev Learn HTML

ESSENTIAL CONCEPTS

- **Document basics:** <!doctype html>, <html>, <head>, <body>, <title>, lang attribute
- **Text & lists:** <h1>-<h6>, <p>, , , <ul>, <ol>, <li>, <blockquote>, <code>
- **Links & media:** <a> (href, target, rel), <img> (src, alt, width, height, loading), <picture>, srcset, <video>, <audio>
- **Tables:** <table>, <thead>, <tbody>, <tr>, <th>, <td>, colspan, rowspan, <caption>
- **Forms:** <form>, <input> types, <label>, <textarea>, <select>, <button>, built-in validation (required, pattern, min, max)
- **Semantic layout:** <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>
- **Interactive elements:** <details>, <summary>, <dialog>, popover attribute
- **Meta & SEO:** <meta> charset, viewport, description, Open Graph tags, canonical links
- **Accessibility:** heading order, alt text, label association, landmarks, ARIA only when native HTML is not enough

CSS

CSS controls how everything looks and adapts. Modern CSS is far more powerful than older tutorials suggest: layout, theming and even simple logic now work without preprocessors or hacks.

TIME REQUIRED	PHASE / LEVEL
4 weeks	**Beginner**

LEARN WITH MDN CSS guide · web.dev Learn CSS

ESSENTIAL CONCEPTS

- **Selectors & cascade:** element, class, id, attribute, pseudo-classes, pseudo-elements, specificity, inheritance, @layer
- **Box model:** content, padding, border, margin, box-sizing, margin collapse
- **Display & positioning:** block, inline, inline-block, static, relative, absolute, fixed, sticky, z-index
- **Flexbox:** flex-direction, justify-content, align-items, flex-wrap, gap, flex-grow/shrink/basis
- **Grid:** grid-template-columns/rows, fr unit, minmax(), auto-fit/auto-fill, grid areas, subgrid
- **Units & sizing:** px, rem, em, %, vh/vw, dvh, clamp(), min(), max(), aspect-ratio

© 2026 Mindflare · IT Department Society

- **Custom properties:** CSS variables, theming, design tokens, color-scheme, dark mode with prefers-color-scheme
- **Modern CSS:** native nesting, :has(), :is(), :where(), container queries, logical properties, color-mix(), view transitions
- **Responsive design:** mobile-first, media queries, fluid typography, responsive images
- **Motion:** transitions, transforms, @keyframes, scroll-driven animations, prefers-reduced-motion
- **Typography & color:** font-face, variable fonts, line-height, hex, rgb, hsl, oklch

> MINDFLARE TIP
> Sass is optional today. Native CSS now has variables and nesting. Learn Sass only if a project you join already uses it.

JavaScript

JavaScript is the one language that runs in the browser and on the server, and it powers nearly every AI web SDK. This is the most important skill in the roadmap, so take your time and build many small projects.

TIME REQUIRED	PHASE / LEVEL
8 weeks	**Beginner**

LEARN WITH javascript.info · MDN JavaScript guide

ESSENTIAL CONCEPTS

- **Variables & types:** let, const, scope, hoisting, primitives, objects, typeof, null vs undefined, Symbol, BigInt
- **Equality & coercion:** ==, ===, Object.is, truthy/falsy, type conversion
- **Control flow:** if/else, switch, for, for...of, for...in, while, break, continue, ternary
- **Functions:** declarations, expressions, arrow functions, default and rest parameters, closures, higher-order functions, this binding
- **Arrays & objects:** map, filter, reduce, find, some, every, flat, at, toSorted; destructuring, spread, optional chaining (?.), nullish coalescing (??)
- **Prototypes & classes:** prototype chain, class syntax, inheritance, private fields (#), getters/setters, static members
- **Collections:** Map, Set, WeakMap, WeakSet, JSON, structuredClone
- **Error handling:** try/catch/finally, throw, custom Error classes
- **Asynchronous JS:** event loop, call stack, microtasks, callbacks, Promises, async/await, Promise.all/allSettled/race, AbortController
- **DOM & events:** querySelector, createElement, classList, event delegation, bubbling and capturing, forms, IntersectionObserver
- **Working with APIs:** fetch, JSON, HTTP errors, loading and error states, CORS basics
- **Browser storage:** localStorage, sessionStorage, cookies, IndexedDB overview
- **Modules & tooling:** ES modules (import/export), npm packages, Vite, ESLint, Prettier
- **Iterators & generators:** Symbol.iterator, generator functions, async iterators

© 2026 Mindflare · IT Department Society

PROJECT IDEAS

Build every one of these without a framework. Frameworks make sense only after you feel the pain they remove.

01 Todo List

Add, delete and complete tasks with persistence.

- Input for new tasks
- Checkbox to mark complete, delete button
- Save to localStorage
- Filter by all / active / done

02 Weather App

Fetch live data from a public weather API.

- Search by city name
- Show temperature, description and icon
- Graceful errors for bad input or network failure
- Loading indicator

03 Image Carousel

Practise timers, events and DOM updates.

- Auto-advance with a timer
- Previous / next buttons and keyboard support
- Dot indicators
- Responsive and accessible

04 Quiz App

State management without a framework.

- One question at a time
- Highlight correct and wrong answers
- Running score, final result and restart
- Questions loaded from a JSON file

05 Typing Speed Test

Timers, string comparison and statistics.

- Random passage selection
- Live WPM and accuracy
- Highlight mistakes as you type
- Local high-score history

© 2026 Mindflare · IT Department Society

Data Structures, Algorithms & Competitive Programming

This is the heart of Mindflare. DSA sharpens problem-solving, makes you fast at turning ideas into code, and is tested in almost every tech interview. Start in Phase 1 and never stop: a few problems every week beat a cram session.

TIME REQUIRED	PHASE / LEVEL
8 weeks core, then ongoing	**Parallel track**

LEARN WITH Codeforces · LeetCode · CP-Algorithms

ESSENTIAL CONCEPTS

- **Complexity:** time and space, Big-O, amortized analysis, reading constraints to pick an approach
- **Arrays & strings:** two pointers, sliding window, prefix sums, difference arrays, string matching
- **Hashing:** hash maps and sets, frequency counting, anagrams, subarray-sum patterns
- **Searching & sorting:** binary search (including on the answer), merge sort, quick sort, counting sort, custom comparators
- **Linear structures:** linked lists, stacks, queues, deques, monotonic stack and queue
- **Recursion:** backtracking, subsets, permutations, N-Queens, divide and conquer
- **Trees:** binary trees, BST, traversals, heaps and priority queues, tries, segment trees, Fenwick trees
- **Graphs:** BFS, DFS, shortest paths (Dijkstra, Bellman-Ford), union-find, topological sort, minimum spanning tree
- **Dynamic programming:** memoization vs tabulation, 1D/2D DP, knapsack, LIS, LCS, DP on trees and bitmasks
- **Greedy & math:** interval scheduling, exchange arguments, GCD, modular arithmetic, sieve, combinatorics
- **Bit manipulation:** masks, XOR tricks, subset enumeration

MINDFLARE TIP

Keep a second language for contests. JavaScript is fine for learning DSA, but C++, Java or Python are the usual choices on Codeforces and CodeChef because of speed and library support. Join the Mindflare weekly contests and track your progress on the leaderboard.

HABIT

Learn by pattern, not by problem count. After each solved problem, write the pattern name and the one insight that cracked it.

HABIT

Always try for 30-45 minutes before reading an editorial, then re-solve the problem from scratch the next day.

HABIT

Practise without AI assistance. You are training your own brain, and most contests and interviews do not allow it.

© 2026 Mindflare · IT Department Society

PART

02

Modern Front-end

Move from pages to applications. TypeScript brings safety, React brings components, Tailwind brings speed, and performance and accessibility make your work professional.

IN THIS PART

TypeScript	3 weeks
React	8 weeks
Tailwind CSS	3 weeks
Web Performance & Accessibility	Ongoing

© 2026 Mindflare · IT Department Society

TypeScript

TypeScript adds static types to JavaScript so many bugs are caught while you type instead of in production. It is now the default for serious projects, and AI SDKs, validation libraries and frameworks are all written for it first.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Intermediate**

LEARN WITH TypeScript Handbook · Total TypeScript (free tutorials)

ESSENTIAL CONCEPTS

- **Basic types:** string, number, boolean, arrays, tuples, enums vs const objects, any, unknown, never, void
- **Type inference:** let the compiler infer; annotate function boundaries and public APIs
- **Object types:** interfaces vs type aliases, optional and readonly properties, index signatures
- **Unions & narrowing:** union and intersection types, literal types, type guards, discriminated unions, exhaustive checks
- **Functions:** typed parameters and returns, overloads, optional and default parameters
- **Generics:** generic functions, classes and constraints, default type parameters
- **Utility types:** Partial, Required, Pick, Omit, Readonly, Record, Exclude, Extract, ReturnType, Awaited
- **Advanced types:** mapped types, conditional types, template literal types, the satisfies operator
- **Configuration:** tsconfig.json, strict mode, path aliases, declaration files and @types packages
- **Runtime validation:** Zod schemas and z.infer to keep types and runtime checks in sync

> MINDFLARE TIP
> Turn on strict mode from day one. Fighting the compiler early teaches you more than any tutorial.

© 2026 Mindflare · IT Department Society

React

React is the most widely used library for building user interfaces. You describe the UI as a function of state, and React updates the screen efficiently. Most AI chat interfaces you have seen are built with it.

TIME REQUIRED	PHASE / LEVEL
8 weeks	**Intermediate**

LEARN WITH react.dev (official docs) · Vite

ESSENTIAL CONCEPTS

- **Core ideas:** components, JSX, props, children, one-way data flow, keys
- **State & events:** useState, controlled inputs, derived state, lifting state up
- **Rendering:** conditional rendering, rendering lists, reconciliation, StrictMode
- **Core hooks:** useEffect (and when NOT to use it), useRef, useReducer, useContext, useMemo, useCallback, custom hooks
- **Modern React (19+):** Actions, useActionState, useOptimistic, use(), ref as a prop, form actions, Server Components overview
- **Forms:** React Hook Form with Zod validation, accessible error messages
- **Server state:** TanStack Query for fetching, caching, mutations and optimistic updates
- **Client state:** Context for light needs; Zustand or Redux Toolkit when state grows
- **Routing:** React Router or TanStack Router, nested routes, route loaders
- **Styling & UI kits:** CSS Modules, Tailwind, shadcn/ui with Radix primitives
- **Performance:** memoization, code splitting with lazy and Suspense, list virtualization, React Compiler awareness
- **Dev workflow:** Vite, React DevTools, error boundaries, ESLint plugin for hooks

PROJECT IDEAS

Ordered from simple to complex. Build each in TypeScript.

01 Todo App

The classic first React project.

- Add, delete and complete tasks
- Filter by status
- Persist to localStorage
- Keyboard-accessible controls

02 Weather App

Practise data fetching and UI states.

- City search with debounced input
- Loading, error and empty states
- Temperature unit toggle
- Cached results with TanStack Query

© 2026 Mindflare · IT Department Society

03 Recipe Finder

Search, list and detail views.

- Search by name or ingredient
- Result cards with images
- Detail route with ingredients and steps
- Favourites saved locally

04 E-commerce Storefront

Shared state across pages.

- Product listing with filters and sort
- Cart with quantity updates and totals
- Persistent cart
- Checkout form with validation

05 Expense Tracker

Forms, derived data and charts.

- Add expenses with category and date
- Filter and search
- Charts by category and month (Recharts)
- Export to CSV

06 Contest Leaderboard UI

A Mindflare-flavoured project.

- Table with live sorting and filtering
- Rank change indicators
- Profile pages with solved-problem heatmap
- Dark and light themes

Tailwind CSS

Tailwind is a utility-first CSS framework: you style elements by composing small classes directly in your markup. It keeps design consistent, removes naming debates, and is the styling default across much of the React and Next.js ecosystem.

TIME REQUIRED	PHASE / LEVEL
3 weeks	Intermediate

LEARN WITH Tailwind CSS Docs · shadcn/ui

ESSENTIAL CONCEPTS

- **Utility-first thinking:** why it works, when to extract components instead of @apply
- **Layout:** flex, grid, container, spacing, sizing, positioning, z-index
- **Typography & color:** font utilities, text sizes, color palette, opacity, gradients
- **Responsive & states:** mobile-first breakpoints, hover, focus, active, disabled, group and peer variants
- **Dark mode:** dark: variant, theme tokens, respecting system preference
- **Configuration (v4):** CSS-first setup with @theme, custom colors, fonts and breakpoints, plugins
- **Component patterns:** clsx, tailwind-merge, class-variance-authority for variant-based components
- **Design systems:** tokens, spacing scale, consistent radius and shadows, building a small reusable UI kit

© 2026 Mindflare · IT Department Society

Web Performance & Accessibility

Fast and accessible sites reach more people and rank better. These are not extras; they are marks of a professional. Practise them continuously instead of studying them once.

TIME REQUIRED	PHASE / LEVEL
Ongoing	**All levels**

LEARN WITH web.dev · WCAG Quick Reference

ESSENTIAL CONCEPTS

- **Core Web Vitals:** LCP (loading), INP (responsiveness), CLS (layout stability) and how to measure them
- **Loading performance:** image formats (WebP, AVIF), lazy loading, font loading, code splitting, caching headers, CDNs
- **Rendering performance:** avoiding layout thrash, long tasks, web workers, virtualization
- **Tools:** Lighthouse, Chrome DevTools Performance panel, WebPageTest, bundle analyzers
- **Accessibility basics:** semantic HTML, keyboard navigation, focus management, color contrast, alt text
- **Testing accessibility:** screen readers (NVDA, VoiceOver), axe DevTools, eslint-plugin-jsx-a11y
- **Inclusive design:** reduced motion, text resizing, touch targets, readable forms and errors

© 2026 Mindflare · IT Department Society

PART

03

Back-end Development

Build the engine behind every product: servers, APIs, databases, authentication and security. You will stay in JavaScript and TypeScript, so you can focus on concepts instead of new syntax.

IN THIS PART

Node.js	2 weeks
Express.js & REST API Design	4 weeks
PostgreSQL & SQL	5 weeks
MongoDB & Redis	3 weeks (2 MongoDB, 1 Redis)
Authentication & Security	4 weeks

© 2026 Mindflare · IT Department Society

Node.js

Node.js runs JavaScript outside the browser using Chrome's V8 engine. Its non-blocking, event-driven design makes it excellent for APIs, real-time features and AI streaming responses.

TIME REQUIRED	PHASE / LEVEL
2 weeks	Intermediate

LEARN WITH Node.js Learn · Node.js API docs

ESSENTIAL CONCEPTS

- **How Node works:** V8, libuv, the event loop, thread pool, blocking vs non-blocking code
- **Modules:** ES modules vs CommonJS, package.json, exports, workspaces
- **Core modules:** fs/promises, path, os, http, events, crypto, url, child_process
- **Streams & buffers:** readable, writable and transform streams, piping, backpressure
- **Concurrency:** worker_threads, cluster, when CPU-heavy work should leave the main thread
- **Modern built-ins:** global fetch, the node:test runner, --watch mode, --env-file, running TypeScript with tsx or native type stripping
- **Tooling:** npm vs pnpm, scripts, lockfiles, npx, nodemon alternatives, debugging with the inspector
- **Configuration:** environment variables, .env files, config validation with Zod
- **Other runtimes:** awareness of Bun and Deno and why they exist

Express.js & REST API Design

Express is the minimal, battle-tested web framework for Node. Pair it with solid REST design and you can build the API behind almost any app. Fastify, Hono and NestJS are worth knowing exist, but the core ideas transfer directly.

TIME REQUIRED	PHASE / LEVEL
4 weeks	Intermediate

LEARN WITH Express docs · MDN Server-side guide

ESSENTIAL CONCEPTS

- **Express basics:** app, routers, route params, query strings, request and response objects (Express 5)
- **Middleware:** ordering, built-in parsers, custom middleware, error-handling middleware, async error handling
- **Project structure:** routes, controllers, services, repositories; separating HTTP concerns from business logic
- **Validation:** request validation with Zod, consistent error responses
- **REST design:** resources, HTTP verbs, status codes, idempotency, pagination, filtering, sorting, versioning
- **API documentation:** OpenAPI / Swagger, Postman or Bruno collections
- **Cross-cutting concerns:** CORS, helmet, rate limiting, compression, request logging with pino
- **File handling:** multipart uploads with multer, streaming files, object storage (S3-compatible)

© 2026 Mindflare · IT Department Society

- **Real-time:** WebSockets with ws or Socket.IO, Server-Sent Events (essential for streaming AI output)
- **Background work:** job queues with BullMQ, cron jobs, webhooks
- **Testing:** unit and integration tests with Vitest and Supertest
- **Alternatives:** Fastify, Hono, NestJS, tRPC: when and why teams pick them

PROJECT IDEAS

Ordered from simple to complex. Every project should include authentication, validation and tests.

01 Personal Blog API

CRUD with users and posts.

- Signup, login, logout
- Create, read, update, delete posts
- Profile endpoints
- Pagination and search

02 To-Do List API

Tasks with categories.

- User-owned tasks only (authorization)
- Categories, labels and due dates
- Filtering and sorting
- Soft delete

03 E-commerce API

Products, carts and orders.

- Product CRUD with image upload
- Cart management
- Order creation inside a database transaction
- Payment webhook (Stripe test mode)

04 Event Management API

Events and registrations.

- Create and publish events
- Register and cancel with capacity limits
- Attendee list with CSV export
- Email reminders via a queue

05 Social Media API

Relationships and feeds.

- Posts, likes and comments
- Follow and unfollow
- Personalized feed query
- Notifications

06 Job Board API

Companies, jobs and applicants.

- Role-based access (recruiter, candidate)
- Job search with filters
- Resume upload
- Application status tracking

© 2026 Mindflare · IT Department Society

07 Real-Time Chat API

Rooms and live messages.

- Rooms and direct messages
- WebSocket delivery with presence
- Message history with cursor pagination
- Typing indicators

08 Contest Platform API

A Mindflare-style judge backend.

- Problems, contests and submissions
- Queue-based code evaluation worker
- Live leaderboard with Redis sorted sets
- Rate limiting per user

PostgreSQL & SQL

Relational databases store structured data in tables and guarantee correctness through constraints and transactions. PostgreSQL is a powerful open-source choice, and its pgvector extension will let you do AI vector search later without adding another database. Everything here transfers to MySQL.

TIME REQUIRED	PHASE / LEVEL
5 weeks	**Intermediate**

LEARN WITH PostgreSQL Tutorial · PG Exercises · Prisma docs

ESSENTIAL CONCEPTS

- **Querying:** SELECT, WHERE, AND/OR/NOT, IN, BETWEEN, LIKE/ILIKE, IS NULL, ORDER BY, LIMIT/OFFSET, DISTINCT
- **Joins:** inner, left, right, full outer, self joins, cross joins, UNION
- **Aggregation:** COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING, subqueries
- **Advanced SQL:** CTEs (WITH), window functions (ROW_NUMBER, RANK, LAG, LEAD), recursive queries
- **Data changes:** INSERT, UPDATE, DELETE, UPSERT (ON CONFLICT), RETURNING
- **Schema design:** tables, data types, primary and foreign keys, constraints, one-to-many, many-to-many, normalization
- **Indexes:** B-tree, composite and partial indexes, GIN for JSONB, EXPLAIN ANALYZE, avoiding N+1 queries
- **Transactions:** ACID, BEGIN/COMMIT/ROLLBACK, isolation levels, locking, deadlocks
- **Programmability:** views, materialized views, functions, triggers
- **JSONB:** storing and querying semi-structured data inside Postgres
- **Migrations & ORMs:** schema migrations, Prisma or Drizzle ORM, type-safe queries, connection pooling
- **Operations:** backups, roles and privileges, managed hosts such as Neon, Supabase and RDS

© 2026 Mindflare · IT Department Society

PROJECT IDEAS

Design the schema first, then write the queries. Draw an ER diagram before you code.

01 Library Management

Books, authors, members and loans.

- Borrow and return flow
- Available books query
- Overdue report
- Most borrowed books

02 Student Grades

Students, courses and marks.

- Enrolment many-to-many
- Average grade per student
- Top scorer per course using window functions
- Semester GPA view

03 Inventory System

Products, suppliers and stock.

- Stock movements ledger
- Low-stock alerts
- Supplier lookup per product
- Trigger to keep stock consistent

04 Online Store

Customers, orders and payments.

- Orders with line items
- Total sales per period
- Best-selling products
- Transactional checkout

05 Contest Leaderboard

Ranking with SQL.

- Submissions and verdicts tables
- Rank users with RANK() and tie-breaks
- Penalty time calculation
- Per-problem solve percentages

© 2026 Mindflare · IT Department Society

MongoDB & Redis

Not everything fits neatly in tables. MongoDB stores flexible JSON-like documents, and Redis keeps data in memory for lightning-fast access. Know what each is best at so you can choose well. If you are short on time, learn Redis first; it appears in more real systems.

TIME REQUIRED	PHASE / LEVEL
3 weeks (2 MongoDB, 1 Redis)	**Intermediate**

LEARN WITH MongoDB University · Redis University

ESSENTIAL CONCEPTS

- **MongoDB basics:** documents, collections, BSON types, SQL vs NoSQL trade-offs
- **CRUD:** insertOne/Many, find, updateOne/Many, deleteOne/Many, bulkWrite
- **Operators:** $eq, $gt, $lt, $in, $and, $or, $exists, $regex, $set, $push, $pull
- **Aggregation:** $match, $group, $sort, $project, $lookup, $unwind, $facet
- **Modeling:** embedding vs referencing, schema validation, Mongoose ODM
- **Indexes:** single field, compound, text, TTL, explain plans
- **Redis data types:** strings, hashes, lists, sets, sorted sets, streams
- **Caching patterns:** cache-aside, TTLs, invalidation, preventing cache stampedes
- **Redis use cases:** sessions, rate limiting, leaderboards with sorted sets, pub/sub, job queues

Authentication & Security

Every real application must know who the user is and protect their data. Security mistakes are expensive, and many are easy to prevent when you know the common patterns.

TIME REQUIRED	PHASE / LEVEL
4 weeks	**Intermediate**

LEARN WITH OWASP Top 10 · MDN Web Security

ESSENTIAL CONCEPTS

- **Passwords:** hashing with argon2 or bcrypt, salting, password reset flows, never storing plain text
- **Sessions vs tokens:** cookie sessions, JWT access and refresh tokens, token rotation and revocation
- **Cookies:** HttpOnly, Secure, SameSite, domain and path scoping
- **OAuth 2.0 & OIDC:** Sign in with Google or GitHub, authorization code flow with PKCE
- **Authorization:** role-based and permission-based access control, ownership checks
- **Modern auth:** passkeys and WebAuthn, magic links, MFA; libraries such as Auth.js, Better Auth and Passport
- **OWASP Top 10:** injection, broken access control, XSS, CSRF, SSRF, insecure design, vulnerable dependencies
- **Browser defences:** CORS, Content Security Policy, security headers with helmet
- **Hardening:** input validation, parameterized queries, rate limiting, secrets management, npm audit

© 2026 Mindflare · IT Department Society

- **Privacy basics:** minimising data, encryption in transit and at rest, handling PII responsibly

> **MINDFLARE TIP**
> Never trust the client. Validate on the server, check ownership on every request, and keep API keys out of front-end code. This rule matters even more when you add AI features.

PART

04

Full-Stack Engineering

Combine front-end and back-end into complete products. Learn to test with confidence, build with Next.js, and ship with containers and automated pipelines.

IN THIS PART

Automated Testing	4 weeks
Next.js	6 weeks
Docker, CI/CD & Deployment	3 weeks

© 2026 Mindflare · IT Department Society

Automated Testing

Tests let you change code without fear. They are also how you prove that prompts, APIs and UI behave correctly once AI features join the mix. Learn the testing pyramid: many fast unit tests, fewer integration tests, a handful of end-to-end tests.

TIME REQUIRED	PHASE / LEVEL
4 weeks	Advanced

LEARN WITH Vitest · Testing Library · Playwright

ESSENTIAL CONCEPTS

- **Fundamentals:** unit vs integration vs end-to-end, arrange-act-assert, test naming, TDD loop
- **Vitest:** setup, describe/it/expect, matchers (toBe, toEqual, toContain, toThrow, toMatchObject), snapshots
- **Mocks & spies:** vi.fn, vi.mock, fake timers, dependency injection for testability
- **Async code:** testing promises, async/await, retries and timeouts
- **Coverage:** collecting and reading coverage reports; coverage is a signal, not a goal
- **React components:** React Testing Library: queries by role, user-event, testing behaviour not implementation
- **API mocking:** Mock Service Worker (MSW) for front-end tests
- **API testing:** Supertest against Express apps, test databases, seeding and cleanup
- **End-to-end:** Playwright: page objects, locators, traces, running across browsers
- **Quality gates:** accessibility checks with axe, running tests in CI on every pull request

Next.js

Next.js is a full-stack framework built on React. It gives you routing, server rendering, API endpoints and optimizations out of the box, and it is the most common home for AI-powered web apps written in TypeScript.

TIME REQUIRED	PHASE / LEVEL
6 weeks	Advanced

LEARN WITH Next.js Learn · Next.js Docs

ESSENTIAL CONCEPTS

- **App Router:** file-based routing, layouts, nested routes, dynamic segments, route groups, parallel and intercepting routes
- **Server & Client Components:** what runs where, the use client boundary, passing props across it
- **Rendering models:** static rendering, dynamic rendering, streaming with Suspense, partial prerendering
- **Data fetching:** fetching in Server Components, caching and revalidation (this area evolves, so follow the official docs for the current model)
- **Mutations:** Server Actions, form handling, optimistic UI, revalidating data after changes
- **Route Handlers:** building API endpoints, streaming responses (key for chatbots), webhooks
- **Middleware:** redirects, auth checks, geolocation, A/B testing at the edge
- **Loading & errors:** loading.js, error.js, not-found.js, error boundaries

© 2026 Mindflare · IT Department Society

- **Database access:** Prisma or Drizzle in server code, connection management in serverless
- **Authentication:** Auth.js or Better Auth, protecting routes and Server Actions
- **SEO & metadata:** Metadata API, Open Graph images, sitemaps, robots
- **Optimization:** next/image, next/font, script loading, bundle analysis
- **Environment & config:** env variables, server-only code, never leaking secrets to the client

PROJECT IDEAS

Each project should be deployed with a live URL and a clear README.

01 Personal Portfolio + Blog

Your public home on the web.

- Static generation from MDX or Markdown
- Dynamic routes for posts
- SEO metadata and Open Graph images
- RSS feed and sitemap

02 E-commerce Store

A complete shop.

- Statically generated product pages
- Cart with Server Actions
- Stripe test-mode checkout
- Admin dashboard to manage products

03 Society Website

A Mindflare-style community site.

- Events and contest pages
- Live leaderboard with revalidation
- Member profiles with GitHub login
- Admin tools for organizers

04 SaaS Dashboard

A multi-user product.

- Authentication and roles
- Charts and data tables
- Subscription plans
- Team invitations by email

© 2026 Mindflare · IT Department Society

Docker, CI/CD & Deployment

Code only matters once real people can use it. Learn to package your app in containers, automate checks with pipelines, and deploy to the cloud with confidence.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Advanced**

LEARN WITH Docker Get Started · GitHub Actions Docs

ESSENTIAL CONCEPTS

- **Containers:** images vs containers, Dockerfile, multi-stage builds, layers and caching, .dockerignore
- **Docker Compose:** running app + Postgres + Redis locally with one command, volumes, networks
- **CI with GitHub Actions:** workflows, jobs, caching, running lint, type-check, tests and build on every pull request
- **CD:** preview deployments, production deploys, environment secrets, rollbacks
- **Hosting options:** Vercel and Cloudflare for front-ends and edge, Render, Railway or Fly.io for servers, AWS for deeper control
- **Web servers:** Nginx as reverse proxy, HTTPS with Let's Encrypt, domains and DNS records
- **Observability:** structured logs, error tracking (Sentry), uptime monitoring, basic metrics
- **Configuration:** twelve-factor app principles, secrets management, environment parity

© 2026 Mindflare · IT Department Society

PART

05

AI for Web Developers

The newest and fastest-moving layer of web development. Learn to add language models, chatbots, retrieval-augmented generation (RAG), agents and machine-learning models to real products, while keeping them reliable, safe and affordable.

IN THIS PART

AI-Assisted Development	Ongoing
LLM Fundamentals & APIs	3 weeks
Building AI Chatbots	2 weeks
Embeddings, Vector Search & RAG	4 weeks
Agents, Tool Use & MCP	2 weeks
ML Models in Websites	3 weeks
Production AI: Evals, Safety & Cost	2 weeks

© 2026 Mindflare · IT Department Society

AI-Assisted Development

AI coding tools can speed you up enormously or quietly stop you from learning. This topic is about using them like a good senior engineer would: as a fast but fallible collaborator.

TIME REQUIRED	PHASE / LEVEL
Ongoing	**All levels**

LEARN WITH GitHub Copilot docs · Claude Code docs

ESSENTIAL CONCEPTS

- **Tool types:** inline completion, chat assistants, IDE agents, terminal coding agents (e.g. Copilot, Cursor, Claude Code)
- **Good workflow:** write the plan yourself, ask for small diffs, review every line, run the tests, commit in small steps
- **Prompting for code:** give context (files, errors, constraints), state acceptance criteria, ask for explanation before code
- **Learning mode:** ask the tool to explain, quiz you and critique your solution instead of writing it
- **Verification:** tests, type checks, linters and code review catch confident-sounding mistakes
- **Risks:** hallucinated APIs, outdated patterns, insecure code, leaked secrets in prompts, licence issues

MINDFLARE TIP

Rule of thumb: learn the fundamentals by hand first. In DSA practice, contests and early projects, work without AI. Use it freely once you can judge its output.

LLM Fundamentals & APIs

Large language models (LLMs) predict text one token at a time, yet that simple mechanism powers chat, summarization, extraction, coding and reasoning. Learn how they behave and how to call them from your own servers.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Advanced**

LEARN WITH Vercel AI SDK · Anthropic docs · OpenAI docs

ESSENTIAL CONCEPTS

- **How LLMs work:** tokens, embeddings, attention (conceptual), context window, temperature and top-p, why models hallucinate
- **Model landscape:** hosted frontier models (Claude, GPT, Gemini), open-weight models (Llama, Mistral, Qwen, Gemma), small vs large, reasoning models
- **Calling models:** official SDKs, the messages format (system, user, assistant), stateless requests, error handling, retries with backoff
- **Unified SDKs:** Vercel AI SDK for provider-agnostic streaming and tools in TypeScript
- **Prompt engineering:** clear instructions, role and context, few-shot examples, delimiters and XML tags, asking for step-by-step reasoning, prompt versioning

© 2026 Mindflare · IT Department Society

- **Structured output:** JSON mode and schemas, Zod validation of model responses, repair and retry loops
- **Tool / function calling:** describing tools with schemas, letting the model request actions, returning results
- **Streaming:** Server-Sent Events, token streaming to the UI, cancellation with AbortController
- **Multimodal:** sending images and PDFs, vision tasks, speech-to-text and text-to-speech APIs
- **Local models:** Ollama and LM Studio for private, offline development
- **Cost & limits:** token pricing, prompt caching, batching, rate limits, choosing the smallest model that works
- **Key safety:** API keys live only on the server; proxy all model calls through your own back-end

PROJECT IDEAS

Warm-up projects before building full chatbots.

01 Prompt Playground

Compare behaviour across settings.

- Run one prompt on several models side by side
- Sliders for temperature and max tokens
- Streaming output
- Token and cost estimate per run

02 Structured Extractor

Turn messy text into clean data.

- Paste a resume or email
- Extract fields using a Zod schema
- Validate and retry on bad output
- Export as JSON or CSV

Building AI Chatbots

A chatbot is more than a text box. It is a streaming UI, a conversation store, a carefully written system prompt and a set of guardrails working together. Build one end to end and you will understand most AI product patterns.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Advanced**

LEARN WITH AI SDK: Chatbot guide · Next.js Learn

ESSENTIAL CONCEPTS

- **Architecture:** browser UI to your API route to the model provider, streaming tokens back
- **Chat UI patterns:** streaming text, markdown and code rendering, auto-scroll, stop and regenerate, copy, feedback buttons, loading skeletons
- **React tooling:** the useChat hook from the AI SDK, optimistic messages, error states
- **Persistence:** database schema for users, conversations and messages; resuming chats; titles generated by the model
- **Context management:** conversation history limits, sliding windows, summarizing older turns, token counting
- **System prompts:** persona, tone, scope, refusal rules, output format; testing prompts against edge cases

© 2026 Mindflare · IT Department Society

- **Memory:** short-term (current chat) vs long-term (user preferences stored in a database)
- **Guardrails:** input and output moderation, topic restriction, length limits, safe handling of markdown and links
- **Rich interactions:** tool calls rendered as UI components, file uploads, suggested follow-ups
- **Voice (optional):** Web Speech API and real-time voice APIs
- **Operations:** rate limiting per user, usage quotas, graceful provider failures and fallbacks

PROJECT IDEAS

Build these in order; each adds one new idea.

01 Streaming Chat Assistant

The foundation project.

- Next.js + AI SDK + Postgres
- Persistent multi-conversation history
- Markdown and code highlighting
- Stop, regenerate and copy buttons

02 Study Buddy Bot

A focused tutor.

- System prompt that teaches by asking questions
- Subject and difficulty selectors
- Quiz mode with structured JSON questions
- Saves weak topics per user

Embeddings, Vector Search & RAG

Models only know what they were trained on and cannot see your private data. Retrieval-Augmented Generation (RAG) fixes this: find the most relevant pieces of your documents first, then give them to the model as context so it can answer with grounded, citeable facts.

TIME REQUIRED	PHASE / LEVEL
4 weeks	**Advanced**

LEARN WITH pgvector · Hugging Face Learn · Ragas

ESSENTIAL CONCEPTS

- **Why RAG:** fresh and private knowledge, fewer hallucinations, citations, cheaper than fine-tuning for most knowledge tasks
- **The pipeline:** ingest, parse, chunk, embed, store, retrieve, (rerank), build prompt, generate, cite
- **Embeddings:** text as vectors, semantic similarity, cosine similarity vs dot product, choosing an embedding model and dimension
- **Vector stores:** pgvector (use Postgres you already know), Qdrant, Chroma, Pinecone, Weaviate, MongoDB Atlas Vector Search
- **Indexing:** exact vs approximate nearest neighbour, HNSW and IVF indexes, recall vs speed trade-offs
- **Document processing:** PDF, HTML, Markdown and code parsing, OCR for scans, cleaning and metadata extraction

- **Chunking strategies:** fixed size with overlap, recursive splitting, structure-aware (headings, functions), semantic chunking, parent-child chunks
- **Better retrieval:** metadata filters, hybrid search (keyword BM25 + vectors), reciprocal rank fusion, reranking with cross-encoders, multi-query, query rewriting, HyDE, contextual retrieval
- **Prompting for RAG:** answer only from context, say "I don't know", cite sources, handling conflicting passages
- **Evaluation:** golden question sets, retrieval metrics (recall@k, MRR), answer faithfulness and relevance, tools like Ragas, regression tests when you change chunking or prompts
- **Failure modes:** bad chunks, stale indexes, the lost-in-the-middle effect, prompt injection hidden in documents
- **Frameworks:** LangChain.js, LlamaIndex.TS, Mastra; build the raw pipeline once before adopting one

> MINDFLARE TIP
> Build your first RAG system with only three ingredients: a Postgres table with pgvector, an embeddings API call and a chat completion. You will understand every moving part.

PROJECT IDEAS

Ordered from simple to complex.

01 DocuChat

Chat with your PDFs.

- Upload and parse PDFs
- Chunk, embed and store in pgvector
- Answers with page-level citations
- Show the retrieved passages for transparency

02 Semantic Problem Search

Find similar coding problems by meaning.

- Embed problem statements
- Search by description, not keywords
- Filter by tag and difficulty
- Hybrid keyword + vector ranking

03 DSA Mentor (Mindflare idea)

A RAG tutor over editorials and notes.

- Index editorials, notes and past contests
- Gives progressive hints instead of full solutions
- Cites the source note for each hint
- Evaluation set of 50 real student questions

04 Company Knowledge Assistant

Production-style RAG.

- Multi-document-type ingestion pipeline
- Per-user access control on retrieved chunks
- Reranking and query rewriting
- Dashboard of retrieval quality metrics

© 2026 Mindflare · IT Department Society

Agents, Tool Use & MCP

An agent is a model that can decide to call tools, observe the results and keep going until a task is done. Used well, agents turn chatbots into assistants that actually do things, but they need careful limits.

TIME REQUIRED	PHASE / LEVEL
2 weeks	Advanced

LEARN WITH Model Context Protocol · AI SDK Agents

ESSENTIAL CONCEPTS

- **The agent loop:** model proposes a tool call, your code runs it, result goes back, repeat until finished or a step limit is hit
- **Workflows vs agents:** fixed pipelines (prompt chaining, routing, parallelization) are simpler and more reliable than free-roaming agents; start there
- **Designing tools:** clear names and descriptions, narrow inputs validated with Zod, helpful error messages, idempotent actions
- **Model Context Protocol (MCP):** an open standard for exposing tools, resources and prompts to AI apps; build a small MCP server in TypeScript and connect it to a client
- **Planning & memory:** task decomposition, scratchpads, retrieving past context
- **Human in the loop:** approval steps for risky actions such as sending emails, spending money or deleting data
- **Frameworks:** AI SDK agents, LangGraph.js, Mastra, provider agent SDKs
- **Safety:** least-privilege credentials, sandboxed code execution, step and cost limits, audit logs

PROJECT IDEAS

Ordered from simple to complex.

01 Weather + Calendar Assistant

Your first tool-calling app.

- Two or three typed tools
- Model chooses tools from user requests
- Show tool calls in the UI
- Step limit and error handling

02 GitHub Code Review Bot

AI reviews on pull requests.

- GitHub webhook receiver
- Fetch the diff and relevant files
- Post structured review comments
- Skip generated files and secrets

03 Support Agent with MCP

Tools over your own data.

- MCP server exposing order and ticket lookups
- Agent answers customer questions
- Human approval for refunds
- Full audit trail of tool calls

© 2026 Mindflare · IT Department Society

ML Models in Websites

Not every smart feature needs an LLM. Classifiers, detectors and recommenders are smaller, faster and cheaper. You can run them in the visitor's browser or serve them from your own back-end. This topic teaches both routes and the minimum machine-learning knowledge you need.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Advanced**

LEARN WITH Transformers.js · TensorFlow.js · ONNX Runtime Web

ESSENTIAL CONCEPTS

- **ML basics for web devs:** supervised vs unsupervised learning, features and labels, train/validation/test split, overfitting, accuracy, precision, recall, F1
- **Python essentials:** NumPy, pandas, scikit-learn, Jupyter notebooks; PyTorch overview. Python is the home of ML, so learn enough to read and ship a model
- **Pretrained vs custom:** start with pretrained models from the Hugging Face Hub; fine-tune only when needed; train from scratch rarely
- **In-browser inference:** Transformers.js, TensorFlow.js, ONNX Runtime Web, MediaPipe; WASM and WebGPU backends; running models in Web Workers
- **Browser trade-offs:** pros: privacy, offline use, no server cost. Cons: download size, slow devices, model licence exposure
- **Server-side inference:** Python microservice with FastAPI serving scikit-learn or PyTorch models, ONNX export, calling it from Node over REST
- **Hosted inference:** Hugging Face Inference Providers, Replicate, Modal, Cloudflare Workers AI; paying per request vs running your own GPU
- **Scaling inference:** batching, caching predictions, background jobs for slow tasks, autoscaling, model versioning and A/B tests
- **Common use cases:** image classification, object detection, sentiment and toxicity detection, speech-to-text (Whisper), OCR, recommendations, anomaly detection, background removal
- **Responsible use:** bias and fairness checks, explaining predictions, consent for camera and microphone, data minimization

PROJECT IDEAS

Ordered from simple to complex.

01 In-Browser Image Classifier

No server required.

- Upload or use the webcam
- Run a pretrained model in a Web Worker
- Show top predictions with confidence
- Works offline as a PWA

02 Sentiment Dashboard

Analyze feedback at scale.

- Paste or import reviews
- Classify with Transformers.js
- Charts of sentiment over time
- Highlight most negative themes

© 2026 Mindflare · IT Department Society

03 Problem Recommender

Suggest the next contest problem.

- Collect solve history
- Train a simple model in scikit-learn
- Serve through a FastAPI endpoint
- Call it from a Node/Next.js app

04 Meeting Notes from Audio

Speech to structured notes.

- Record or upload audio
- Transcribe with Whisper
- Summarize action items with an LLM
- Store searchable notes

Production AI: Evals, Safety & Cost

Demos are easy; reliable AI features are hard. Models are non-deterministic and attackers will try to trick them. Learn to measure quality, defend against abuse and keep your bill under control before you ship.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Advanced**

LEARN WITH OWASP Top 10 for LLM Applications · Langfuse

ESSENTIAL CONCEPTS

- **Evaluations:** golden datasets, automated checks, LLM-as-judge (with human spot checks), regression tests when prompts or models change
- **Observability:** trace every request, log prompts, retrieved chunks, latency, tokens and cost (Langfuse, LangSmith, OpenTelemetry); mind personal data in logs
- **Prompt injection:** direct and indirect attacks, treating all retrieved or user text as untrusted, separating instructions from data
- **Output handling:** never execute or render model output unsafely (sanitize HTML and markdown, validate JSON, check URLs)
- **Data protection:** PII redaction, data retention, provider terms, regional rules, not sending secrets to models
- **Abuse prevention:** authentication, per-user rate limits and quotas, spend caps, CAPTCHA for public endpoints
- **Cost control:** prompt caching, semantic caching, routing easy tasks to small models, limiting context, batching, setting max tokens
- **Reliability:** timeouts, retries, circuit breakers, fallback providers, graceful degradation when the model is down
- **UX honesty:** tell users when content is AI generated, show sources, allow feedback and correction
- **Responsible AI:** bias, fairness, copyright and licensing, human oversight for high-stakes decisions

MINDFLARE TIP

Treat the model like an untrusted but useful intern: give it only the access it needs, check its work, and never let it be the only safeguard.

© 2026 Mindflare · IT Department Society

© 2026 Mindflare · IT Department Society

PART

06

Beyond the Roadmap

Finish by proving your skills. Round out your knowledge with senior-level topics, build capstone projects that tell a story, and prepare for internships and placements with a portfolio that stands out.

IN THIS PART

Design Patterns & Clean Code	2 weeks (after Part 03)
Additional Skills for Senior Levels	Ongoing
Capstone Projects	6-10 weeks each
Portfolio, Resume & Placement Prep	Start early, refine monthly

© 2026 Mindflare · IT Department Society

Design Patterns & Clean Code

Design patterns are proven solutions to common design problems. Many frameworks you use daily are built from them, so learning them explains how those tools work under the hood and helps you write code that others can maintain.

TIME REQUIRED	PHASE / LEVEL
2 weeks (after Part 03)	**Intermediate**

LEARN WITH Refactoring Guru · Patterns.dev

ESSENTIAL CONCEPTS

- **Principles:** SOLID, DRY, KISS, separation of concerns, composition over inheritance, coupling and cohesion
- **Creational patterns:** factory, abstract factory, builder, singleton (and why to be careful), prototype
- **Structural patterns:** adapter, decorator, facade, proxy, composite
- **Behavioral patterns:** strategy, observer, command, state, iterator, template method, chain of responsibility (Express middleware!)
- **Front-end patterns:** container and presentational components, custom hooks, compound components, render props
- **Back-end patterns:** repository, service layer, dependency injection, CQRS overview
- **Code quality:** naming, small functions, code review habits, refactoring safely with tests

Additional Skills for Senior Levels

After your first year or two on the job, these topics separate mid-level developers from senior ones. You do not need them now. Know they exist and learn them when a real problem demands them.

TIME REQUIRED	PHASE / LEVEL
Ongoing	**Advanced**

ESSENTIAL CONCEPTS

- **System design:** load balancing, horizontal scaling, caching layers, CDNs, queues, consistency vs availability, rate limiting, back-of-envelope estimation
- **Architecture:** monolith vs microservices, event-driven design, domain-driven design, CQRS, event sourcing
- **Databases at scale:** query tuning, replication, sharding, connection pooling, read replicas, data migrations
- **API styles:** GraphQL, tRPC, gRPC, WebSockets, webhooks, API versioning strategies
- **Cloud:** AWS, Google Cloud or Azure core services: compute, storage, networking, IAM, serverless
- **Infrastructure:** Kubernetes basics, Terraform / infrastructure as code, observability with logs, metrics and traces
- **Security depth:** threat modelling, secrets rotation, supply-chain security, penetration-testing basics
- **Advanced front-end:** WebAssembly, PWAs and service workers, WebRTC, micro-frontends, design systems, i18n

© 2026 Mindflare · IT Department Society

- **Edge & serverless:** Cloudflare Workers, edge functions, cold starts, cost trade-offs
- **AI depth:** fine-tuning and distillation, LoRA, evaluation at scale, GPU serving (vLLM), multi-agent systems, speech and vision pipelines
- **Monorepos & tooling:** Turborepo, Nx, build caching, shared packages
- **Soft skills:** writing design docs, estimating work, mentoring, communicating trade-offs

Capstone Projects

Choose one or two and treat them like real products: write a README, deploy them, add tests, measure performance and keep a changelog. Capstones are what interviewers will ask you about.

TIME REQUIRED	PHASE / LEVEL
6-10 weeks each	**Advanced**

PROJECT IDEAS

Ordered by how much of the roadmap each one exercises.

01 ContestHub: A Coding Contest Platform

Full-stack. Next.js, TypeScript, PostgreSQL, Redis, Docker.

- Authentication, roles and contest registration
- Problem statements with Markdown and math
- Submissions judged by an open-source execution engine such as Judge0 running in a sandbox
- Live leaderboard with Redis sorted sets and tie-break rules
- Admin panel for organizers; CI/CD and a public demo

02 AskMindflare: An AI Knowledge Assistant

AI. RAG, chatbot, evaluation and safety.

- Ingest club notes, editorials and FAQs
- Hybrid retrieval, reranking and cited answers
- Streaming chat UI with saved history
- Evaluation set, tracing dashboard and per-user rate limits
- Prompt-injection tests documented in the README

03 SmartLearn: A Personalized Learning Platform

Full-stack + AI + ML.

- Roadmap tracker with progress analytics
- LLM-generated quizzes validated against a schema
- Recommendation model served from a FastAPI service
- In-browser model for a fun feature (for example handwriting or sentiment)
- Notifications and weekly digest through background jobs

© 2026 Mindflare · IT Department Society

Portfolio, Resume & Placement Prep

Skills only help if people can see them. Package your work clearly, practise explaining it, and prepare for the interview formats that campus placements and internships actually use.

TIME REQUIRED	PHASE / LEVEL
Start early, refine monthly	**All levels**

LEARN WITH GitHub Profile README · Roadmap.sh

ESSENTIAL CONCEPTS

- **GitHub:** pinned repositories, consistent commits, clean history, descriptive README files
- **README template:** problem, live demo link, screenshots or GIF, architecture diagram, tech choices and why, how to run, what you would improve
- **Deployment:** every project has a working URL; uptime matters more than features
- **Resume:** one page, three or four strongest projects, results in numbers (users, latency, accuracy, rank), links that work
- **DSA readiness:** patterns over problem count, timed practice, explain your approach aloud, contest ratings as supporting proof
- **CS fundamentals:** operating systems, DBMS, computer networks, OOP, SQL queries: all common in campus interviews
- **System design basics:** design a URL shortener, chat app or feed; discuss trade-offs calmly
- **Talking about AI projects:** what data you used, how you evaluated quality, how you handled cost, failures and security
- **Behavioral rounds:** STAR stories about teamwork, conflict and learning from failure
- **Community:** mock interviews with Mindflare members, open-source contributions, hackathons, tech talks, a LinkedIn profile that mirrors your resume
- **Internships:** apply early and widely, tailor your resume, follow up politely, treat small gigs as real experience

Suggested Timeline

A realistic month-by-month view of the 77-week plan. DSA runs in parallel the whole time with three to five problems per week and a weekly Mindflare contest.

Phase	Weeks	About months	What you finish
1 Foundations	1-16	1-4	Dev tooling, Git, HTML, CSS, JavaScript; 4+ vanilla JS projects; DSA basics (arrays, hashing, sorting)
2 Modern Front-end	17-30	4-7	TypeScript, React, Tailwind; 3 React projects; DSA: trees, recursion, graphs intro
3 Back-end	31-48	7-11	Node, Express, PostgreSQL, Redis, auth and security; 2 API projects; DSA: graphs, DP
4 Full-Stack	49-61	11-14	Testing, Next.js, Docker, CI/CD; first deployed full-stack product
5 AI Track	62-77	14-18	LLM APIs, chatbot, RAG, agents, ML models, production AI; 2 AI projects
6 Launch	ongoing	from month 12	Capstone, portfolio, resume, mock interviews, internship and placement applications

DO NOT WAIT

Start applying for internships once you finish Phase 3 with a deployed project. Add AI skills to your resume as you complete Phase 5. You do not need to be done to begin.

© 2026 Mindflare · IT Department Society

Learning to build is a journey.

Be patient with yourself. Solve one more problem, build one more feature, ship one more project. Consistency beats intensity, and the algorithm that sparks your intuition today is the system you will design tomorrow.

Team Mindflare
IT Department Society

© 2026 Mindflare · IT Department Society